06

AULA 06 · Terça, 27/10

Supply Chain Security: Ataques e Frameworks

Como um atacante compromete seu produto sem tocar no seu código

Módulo · CY-SE-013 - Supply Chain Security

Prof.: Everton J Benedicto

ada

Cronograma das Aulas

01
Sex 16/10
Policy as Code e Compliance Automatizado

02
Seg 19/10
Linguagem Rego e Escrita de Políticas

03
Ter 20/10
Enforcement: OPA Gatekeeper e Admission Control

04
Sex 23/10
SBOM: Fundamentos, SPDX e CycloneDX

05
Seg 26/10
Análise de SBOM e Gestão de Dependências

06
Ter 30/10
Supply Chain Security: Ataques e Frameworks

07
Sex 03/10*
Integridade e Procedência de Artefatos

08
Ter 04/11
Compliance Contínuo, Auditoria e Projeto Final

09
Sex 06/11
Concluir atividades e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!

Câmera ligada
+ mão levantada
= aula top!

INTERVALO · 16h30m

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

ICE BREAKER

Vamos nos conhecer!

Uma pergunta para cada aluno

Quando você projeta o seu futuro e a sua trajetória profissional, o que brilha mais forte no seu horizonte: o desafio de desbravar a profundidade técnica de uma área, ou a missão de inspirar, liderar e multiplicar o potencial de outras pessoas?

Cada um responde, um a um!

Profundidade técnica



Aprofundar em uma área

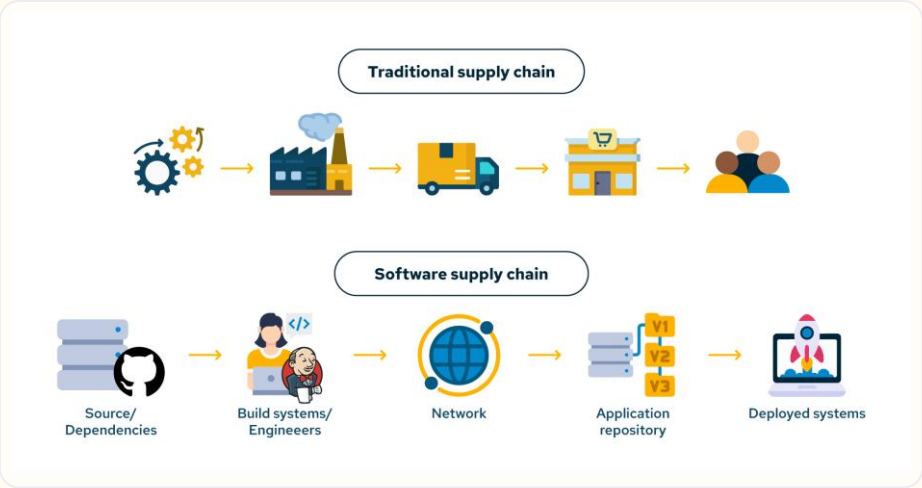
Liderar e inspirar



Inspirar e multiplicar pessoas

MODELO DE AMEAÇA

Três fases de ataque, um risco escondido



1. **Origem:** um commit malicioso, senha vazada ou branch sem revisão abrem a porta.
2. **Build:** typosquatting, dependency confusion ou build poisoning disfarçados.
3. **Distribuição:** artefato trocado, downgrade forçado, chave roubada ou mirror falso.
4. **Confiança transitiva:** você herda a confiança de TUDO que construiu o pacote.

Fonte: clavis.com.br — Ameaças à cadeia de software

FASE DE BUILD

Vetores que exploram dependências

1. **Typosquatting:** nome parecido engana no install (reqests em vez de requests).
2. **Dependency confusion:** pacote público imita o nome interno com versão maior e vence.
3. **Pacote malicioso:** script postinstall rouba variáveis de ambiente ao instalar.
4. **Build poisoning:** plugin do CI comprometido injeta código durante a compilação.
5. **Cache envenenado:** chave previsível e permissão fraca deixam plantar um artefato falso.

> 💡 ANALOGIA
>
> *Dependency confusion é como um entregador que, vendo dois 'João Silva' no prédio, entrega para o que gritar mais alto — o instalador pega a versão 'mais nova', mesmo sendo do atacante.*

ESTUDOS DE CASO

Quatro ataques, quatro lições

1. **SolarWinds / SUNBURST (2020):** update oficial saiu contaminado e assinado — 18 mil vítimas.
2. **Codecov (2021):** script de CI alterado vazou segredos de milhares de empresas por 73 dias.
3. **ua-parser-js (2021):** conta npm roubada infectou 8 milhões de downloads por semana.
4. **XZ Utils (2024):** 2 anos construindo confiança até plantar um backdoor escondido no sshd.

💡 ANALOGIA

XZ Utils é o 'funcionário exemplar' que passa dois anos ganhando confiança para, um dia, deixar a porta dos fundos aberta — e só foi pego porque alguém notou 500 ms a mais no login.

FRAMEWORK DE MATURIDADE

SLSA — Build track

1 · L1 · Proveniência

O build gera um "recibo" de como foi feito — mas esse recibo ainda não tem firma reconhecida (build automatizado CI\CD).

2 · L2 · Assinada

Um serviço de build confiável assina esse recibo — como um cartório, comprova quem emitiu.

3 · L3 · Não falsificável

Cada build roda isolado, numa "sala trancada": não dá pra forjar o recibo de outro artefato.

4 · L4 · Reprodutível

Nível máximo (rascunho da spec): dois builds independentes geram o artefato idêntico, bit a bit.

📖 GLOSSÁRIO

SLSA — Supply-chain Levels for Software Artifacts: a "régua" que mede, em níveis, o quanto dá pra confiar em como um software foi construído.

NIST SP 800-218

SSDF: quatro grupos de práticas

1. **PO — Prepare the Organization:** PO.5.1 — separar bem os ambientes de dev, teste e produção, pra ninguém mexer direto onde não deve
2. **PS — Protect the Software:** PS.2.1 e PS.3.1 — assinar cada versão publicada e garantir que ela chegue intacta, sem alterações no caminho
3. **PW — Produce Well-Secured Software:** PW.4.4 e PW.4.1 — rodar análise estática no código e revisar toda biblioteca de terceiros antes de usar
4. **RV — Respond to Vulnerabilities:** RV.1.3 — cruzar o SBOM com bancos de vulnerabilidades (CVEs) pra saber na hora se algo que você usa foi afetado

💡 ANALOGIA

***SLSA** é a nota do teste de resistência do cofre; **SSDF** é o manual de boas práticas da equipe que cuida do banco. Um mede o artefato, o outro organiza o processo.*

📖 GLOSSÁRIO

SSDF — Secure Software Development Framework
NIST SP 800-218 — documento oficial que define o SSDF (boas práticas de dev seguro).
SBOM — a "lista de ingredientes" do seu software.
CVE — identificador de uma vulnerabilidade já catalogada.

HARDENING

Os quatro vetores, em detalhe

Cada vetor ataca uma etapa diferente do npm install — veja o ataque e a defesa de cada um.

1. **Scripts maliciosos —** hooks preinstall/postinstall executam código ao instalar; ignore-scripts=true bloqueia isso.
2. **Lockfile drift —** npm install pode resolver versões diferentes a cada vez; npm ci trava exatamente o que está no lockfile.
3. **Vulnerabilidades conhecidas (CVE) —** npm audit --audit-level=high barra o pipeline se achar uma falha pública catalogada.
4. **Dependency confusion —** um pacote público pode usar o mesmo nome do seu pacote interno; @empresa:registry=... fixa a origem certa.

GLOSSÁRIO

npm — Node Package Manager — gerenciador de pacotes do Node.js · **CVE** — Common Vulnerabilities and Exposures — catálogo público de vulnerabilidades conhecidas · **CI** — Continuous Integration — testes automáticos a cada push

HARDENING

npm: lockfile, scripts, assinaturas e registry

Mesmo arquivo, quatro cadeados: scripts maliciosos, lockfile, CVEs e dependency confusion.

bash

```
# .npmrc
package-lock=true
ignore-scripts=true          # bloqueia postinstall
audit-level=high
save-exact=true
@empresa:registry=https://nexus.empresa.com.br/repository/npm-internal/
# CI
git diff --exit-code package-lock.json   # lockfile imutável
npm ci --ignore-scripts
npm audit signatures                     # Sigstore
npm audit --audit-level=high
```



GLOSSÁRIO

npm — Node Package Manager — gerenciador de pacotes do Node.js · **CI** — Continuous Integration — testes automáticos a cada push

HARDENING

CI/CD: travando o workflow que empacota tudo

Mesmo com o pacote limpo, um workflow maldotado pode vazar credenciais ou rodar código sem revisão:

yaml

```yaml
# build-hardened.yml
permissions:
  contents: read
  id-token: write          # OIDC: token temporário, sem secret fixo

- uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 # v4.1.1
  with:
    persist-credentials: false   # token não sobrevive após o job

- run: npm audit signatures      # confere a assinatura Sigstore do pacote
```



GLOSSÁRIO

OIDC — credencial temporária emitida só para aquele job, sem senha fixa no repositório · **SHA** — identificador único e imutável de uma versão; travar por SHA evita que a tag mude depois

HARDENING

O mesmo cadeado, em Python e Docker

O princípio se repete fora do npm: trave pelo conteúdo exato, não pela versão ou pelo nome.

bash

```
# Python — hash de cada dependência, não só o número da versão
pip-compile --generate-hashes -o requirements.txt requirements.in
pip install --require-hashes -r requirements.txt

# Docker — imagem-base pelo digest, não pela tag
docker buildx imagetools inspect node:20-bookworm-slim
FROM node:20-bookworm-slim@sha256:2cf067cfed83d5ea958367df9f966191a942351a2df77d6f0193e162b5febfc0

# Docker — Como obter o digest de uma imagem ☺
docker buildx imagetools inspect node:20-bookworm-slim
```



GLOSSÁRIO

digest — impressão digital do conteúdo exato de uma imagem — muda se um único byte mudar · **tag** — apelido de uma imagem (ex: 20-bookworm-slim); pode ser republicado apontando para outro conteúdo · **hash** — valor calculado a partir do conteúdo de um arquivo; --require-hashes só instala se bater

FRONTEIRA NOVA

Riscos emergentes em cadeias de IA

1. **Model poisoning —** dados de treino manipulados ensinam o modelo a errar de propósito; conferir origem, hash e observar o drift pega isso cedo.
2. **Dataset envenenado —** dataset de terceiros pode vir comprometido; um SBOM do dataset com hash mostra se algo mudou.
3. **Backdoor em modelo pré-treinado —** modelo baixado pronto pode esconder um gatilho malicioso; hash dos pesos e atestação de procedência evitam a armadilha.
4. **Dependência de API de modelo —** modelo de terceiro pode mudar de comportamento sem aviso; versionamento, SLA e um fallback protegem o pipeline.
5. **Licenças em embeddings —** modelo também tem licença de uso; inventário no ML-BOM evita dor de cabeça jurídica depois.

GLOSSÁRIO

SBOM — lista de componentes de um software · **ML-BOM** — inventário de modelos e dados de IA usados · **API** — porta de entrada para usar outro serviço · **SLA** — contrato que garante disponibilidade do serviço · **hash** — "impressão digital" do arquivo; muda se ele for alterado · **drift** — quando o modelo muda de comportamento aos poucos, sozinho · **atestação de procedência** — prova verificável de como algo foi criado · **fallback** — plano B se o serviço principal falhar

MÃO NA MASSA

Laboratório: hardening de dependências em um pipeline Node.js

1. **Mapear:** fases source/build/distribution e posicionar os vetores
2. **Casos:** SolarWinds, Codecov, ua-parser-js, XZ — vetor, impacto, controle
3. **Configurar:** .npmrc com ignore-scripts, save-exact e registry
4. **Verificar:** npm ci --ignore-scripts, npm audit signatures, npm audit
5. **Mapear:** cada controle a um nível SLSA e a uma prática SSDF

✅ ENTREGÁVEL

Mapa de ameaças, .npmrc e workflow endurecidos, matriz SLSA/SSDF. Runbook: labs/Aula_06_Runbook.md

RESUMO

Recapitulando a Aula

- A cadeia tem três fases (origem, build, distribuição) e cada uma tem vetores e controles próprios.
- Typosquatting, dependency confusion e build poisoning exploram confiança transitiva.
- SLSA mede garantias de procedência (L1→L4); SSDF organiza práticas (PO, PS, PW, RV).
- Hardening prático: lockfile com hash, scripts bloqueados, assinaturas verificadas, registry autenticado.

Obrigado!

Próxima aula: Integridade e Procedência de Artefatos



Prof.: Everton J Benedicto